International Cybersecurity and Digital Forensics Academy (ICDFA)

CIP-B101: Basic Computer Skills for Digital Forensics

Lab 1A: Number Systems, ASCII, Timestamps and Data Representation

Student Name: Almustapha Yusuf
Student ID: fsdw11343@students.icdfa.edu.ng
Date: August 9, 2026
Platform: Kali Linux (WSL)

A practical foundation for interpreting digital evidence at byte, text, time and memory levels.

Task A: Number Systems and Manual Conversion

Digital forensic examiners routinely encounter hexadecimal values in memory dumps, disk sector editors, file header signatures, hash outputs, and metadata structures. The ability to fluently convert between decimal (base 10), binary (base 2), and hexadecimal (base 16) is therefore a foundational skill. This task demonstrates manual conversion methods and validates the results using Linux shell arithmetic, the bc calculator, and Python 3. Understanding number systems ensures that raw evidence values are correctly interpreted, preventing misidentification of file types, incorrect timestamp readings, or flawed hash comparisons during an investigation.

A.1 Manual Conversion Methods

1. Convert 101010 (binary) to decimal:
Positional weights from right to left: 0x2^0 + 1x2^1 + 0x2^2 + 1x2^3 + 0x2^4 + 1x2^5 = 0 + 2 + 0 + 8 + 0 + 32 = **42**

2. Convert AB2 (hex) to decimal:
A=10, B=11. Weights: 2x16^0 + 11x16^1 + 10x16^2 = 2 + 176 + 2560 = **2738**

3. Convert 126 (decimal) to binary:
Division by 2: 126/2=63 r0, 63/2=31 r1, 31/2=15 r1, 15/2=7 r1, 7/2=3 r1, 3/2=1 r1, 1/2=0 r1. Remainders upward: **1111110**

4. Convert 2738 (decimal) to hex:
Division by 16: 2738/16=171 r2, 171/16=10 r11(B), 10/16=0 r10(A). Remainders upward: **AB2**

5. Convert 101010110010 (binary) to hex:
Group in fours from right: 1010 1011 0010 = A B 2. Result: **AB2**

A.2 Command Validation

```
echo $((2#101010)) # 42 | echo $((16#AB2)) # 2738
echo "obase=2;126" | bc # 1111110 | printf "%X " 2738 # AB2
python3 -c 'print(hex(int(101010110010,2)))' # 0xab2
```

A.3 Evidence Table

Evidence Value	Given Base	Manual Answer	Command Used	Verified?
101010	Binary	42	echo $((2#101010))	Yes
AB2	Hexadecimal	2738	echo $((16#AB2))	Yes
126	Decimal	1111110	bc: obase=2;126	Yes
2738	Decimal	AB2	printf '%X' 2738	Yes
101010110010	Binary	AB2	python3 hex()	Yes

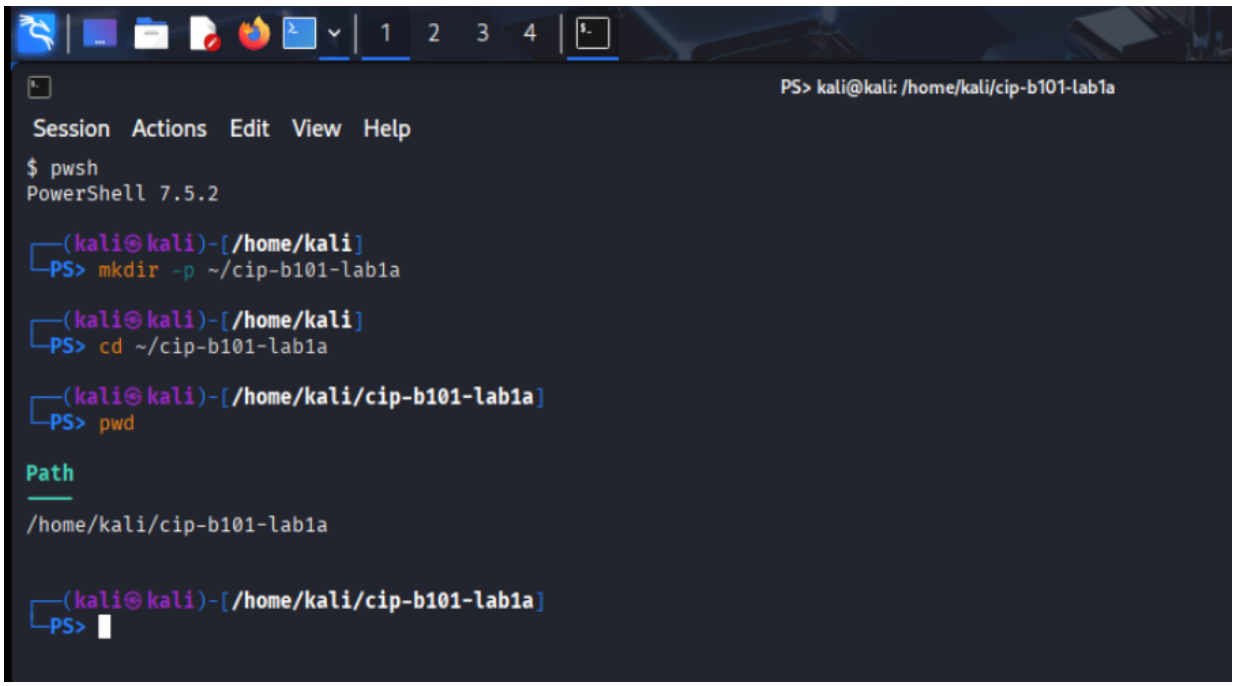

Figure 1: Working folder creation (Screenshot 1)

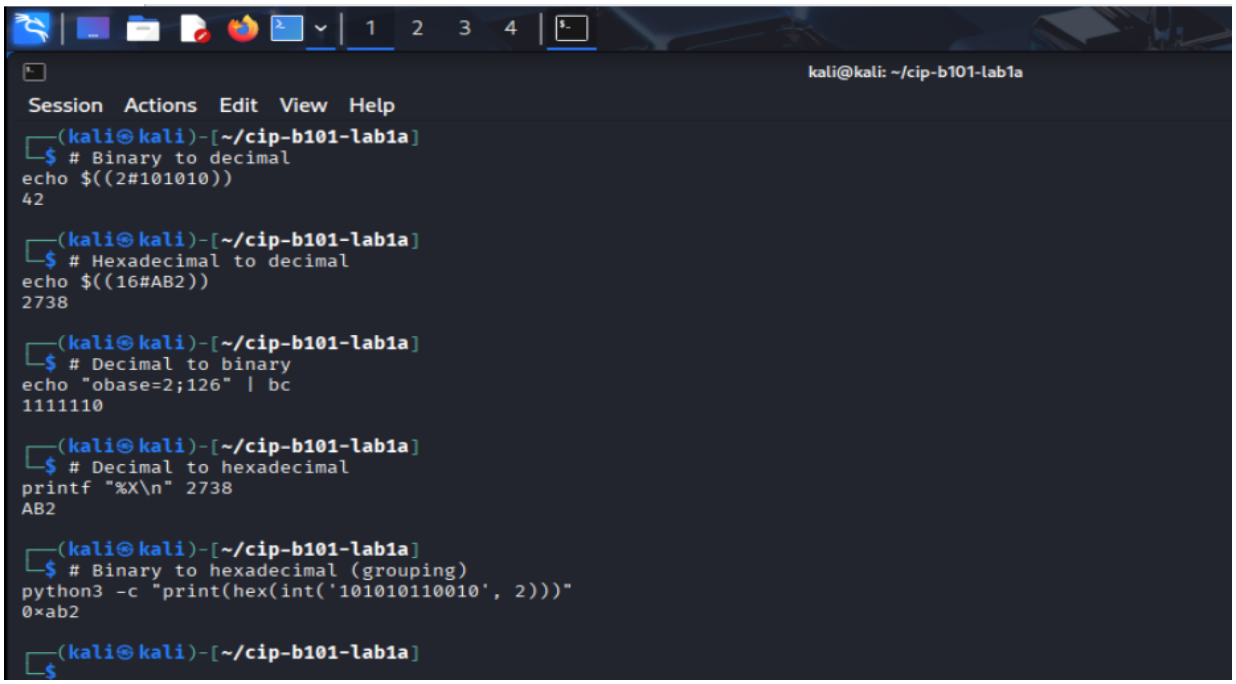

Figure 2: Number system conversion commands (Screenshot 2)

Task B: ASCII, UTF-8 and Hex Evidence Strings

In digital forensics, evidence is frequently stored as raw bytes rather than human-readable text. A single byte can represent an ASCII character, part of a multi-byte UTF-8 character, a numeric value, or opaque binary data. The ability to convert between text and hexadecimal representations is essential when examining file headers, network packets, registry values, and memory dumps. This task demonstrates the use of the xxd utility and Python to perform these conversions, and highlights the importance of understanding byte-level content versus visible text.

B.1 Text to Hex Conversion

The command echo -n "Hello" > evidence_text.txt created a file containing five bytes without a trailing newline. The xxd utility displayed each byte as a two-digit hexadecimal value alongside its ASCII representation. The output confirmed that H=48 (decimal 72), e=65, l=6c, l=6c, o=6f. Using xxd -p produced the plain hex string 48656c6c6f without offsets or ASCII columns, which is the format most commonly encountered in forensic tools and hex editors.

B.2 Hex to ASCII Conversion

The reverse conversion was performed using printf piped through xxd -r -p, which reads a plain hex string and outputs the decoded ASCII bytes. The command printf "48656c6c6f" | xxd -r -p successfully produced "Hello". Python was also used with the binascii module, providing the same results: binascii.hexlify() converted bytes to hex, and binascii.unhexlify() performed the reverse. The ord() and chr() functions demonstrated character-to-integer mapping, with ord(H)=72 and chr(72)=H.

B.3 Named Evidence File

A file named Almustapha_lab1a.txt was created containing "Digital Forensics Begins With Bytes" using echo -n to avoid a trailing newline. The xxd output confirmed 35 bytes of pure ASCII text with no trailing 0a byte. The hex values for each character were verified: D=44, i=69, g=67, i=69, t=74, a=61, l=6c, and so on through the entire string.

B.4 The Newline Byte (0x0A)

When echo is used without the -n flag, it automatically appends a newline character (0x0A) to the output. This means a file created with echo "Hello" contains six bytes (48 65 6c 6c 6f 0a), while echo -n "Hello" produces only five bytes (48 65 6c 6c 6f). In forensic work, this distinction matters because cryptographic hashes are byte-sensitive. The presence or absence of a single trailing newline will produce entirely different hash values, which could lead to incorrect evidence verification if the examiner is unaware of this behavior. An examiner should never assume every hex value represents readable text; some bytes may represent binary structures, images, compressed data, or encrypted content.

Figure 3: ASCII/Hex conversion evidence (Screenshot 3)

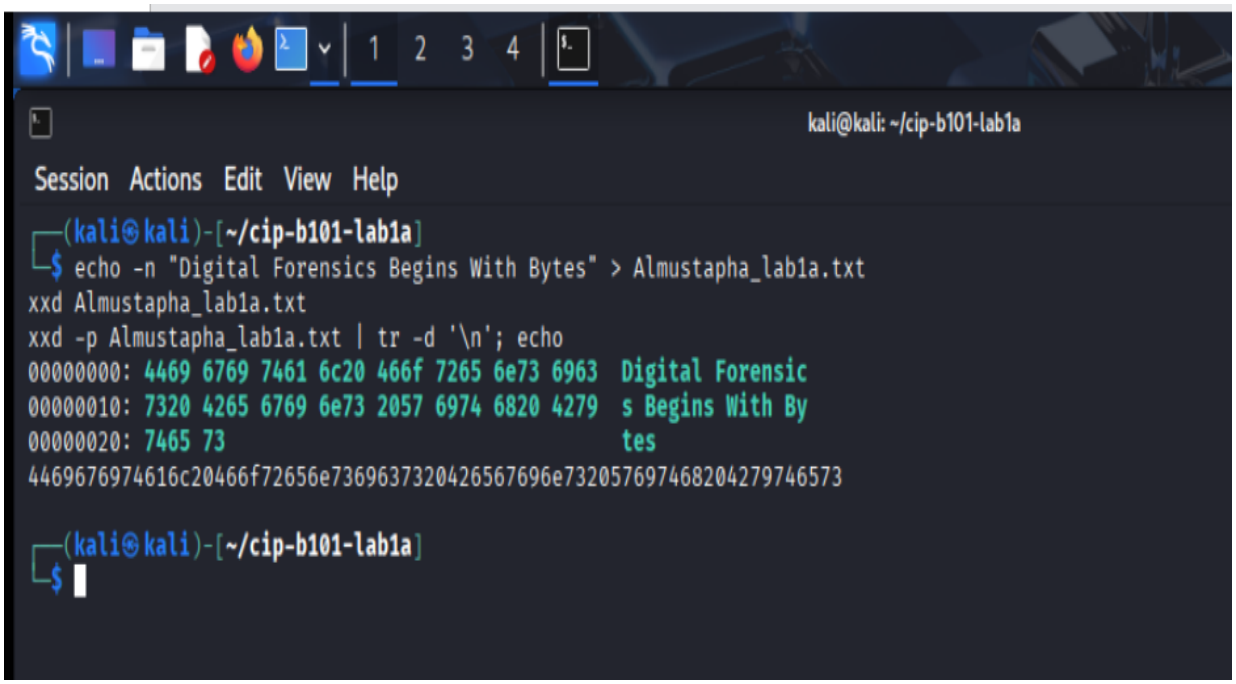

Figure 4: Named evidence file hex dump (Screenshot 4)

Task C: Epoch Time and Forensic Timestamps

Timestamps are central to forensic timelines. Incorrect conversion can place a suspect action at the wrong time, potentially undermining an entire investigation. This task introduces Unix epoch time (seconds since 1970-01-01 00:00:00 UTC) and Mac Absolute Time (seconds since 2001-01-01 00:00:00 UTC), demonstrating how different operating systems and applications encode time values differently. The 978,307,200-second offset between the two epochs is a critical constant that forensic analysts must remember when examining evidence from Apple devices or applications that use the Cocoa Core Data framework.

C.1 Unix Epoch Conversions

The command date +%s returned the current Unix timestamp (1786271224 at the time of execution). Two historical timestamps were then converted to readable UTC format using date -u -d @timestamp. The timestamp 1643643491 corresponded to Mon Jan 31, 15:38:11 UTC 2022, and 1675008832 corresponded to Sun Jan 29, 16:13:52 UTC 2023. These conversions were validated using Python datetime.

C.2 Local Time vs UTC

The system local time was Sun Aug 9, 06:35:47 AM EDT 2026, while UTC showed Sun Aug 9, 10:35:47 AM UTC 2026. This 4-hour difference (EDT is UTC-4) demonstrates why forensic analysts must always state whether their timestamp outputs are in UTC or local time. In formal forensic work, timeline confusion caused by timezone mistakes can seriously weaken the reliability of findings. All timestamp evidence in this report is explicitly stated in UTC.

C.3 Mac Absolute Time Conversion

Mac Absolute Time 725846400 was converted by adding the Cocoa epoch offset of 978307200 seconds, yielding a Unix timestamp of 1704153600. This corresponds to 2024-01-02 00:00:00 UTC, confirming that the value represents exactly midnight on January 2, 2024 in the Mac epoch system. This conversion is essential when analyzing evidence from macOS systems, iOS devices, or applications using Core Data.

C.4 Timestamp Worksheet

Timestamp Value	Format	Conversion Command	Readable UTC Result
1643643491	Unix seconds	date -u -d @1643643491	Mon 2022-01-31 15:38:11 UTC
1675008832	Unix seconds	date -u -d @1675008832	Sun 2023-01-29 16:13:52 UTC
725846400	Mac Absolute Time	python3: +978307200 offset	Tue 2024-01-02 00:00:00 UTC
1786271224	Unix seconds	date +%s	Sun 2026-08-09 10:33:44 UTC

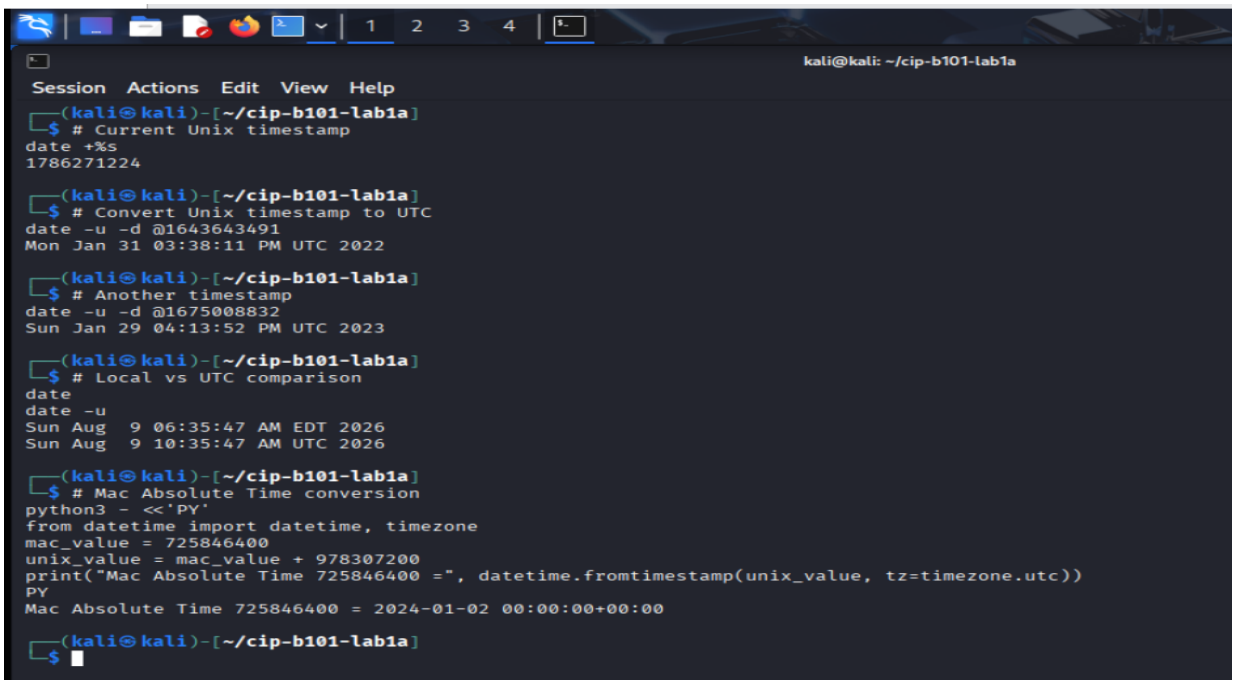

Figure 5: Timestamp conversion outputs (Screenshot 5)

Task D: Hashing and the Newline Problem

Hashing is used to verify that evidence has not changed. Even a single-byte difference produces a completely different hash. This task demonstrates that echo (which appends a newline) and echo -n (which does not) produce files with different byte contents, resulting in entirely different MD5 and SHA-256 hash values. Understanding this behavior is critical for forensic integrity verification.

D.1 Hello File Comparison

hello_with_newline.txt contains 6 bytes: 48 65 6c 6c 6f 0a (the trailing 0a is the newline). hello_without_newline.txt contains only 5 bytes: 48 65 6c 6c 6f. Despite appearing identical when viewed with cat, the xxd output clearly shows the extra byte. The MD5 hashes were completely different: 09f7e02f1290be211da707a266f153b3 (with newline) vs 8b1a9953c4611296a827abf8c4780d47 (without). The SHA-256 hashes were similarly different: 66a045b452102c59d... vs 185f8db32271fe25...

D.2 Name File Comparison

The same test was repeated with the name "Almustapha". name_with_newline.txt (10 bytes with trailing 0a) produced MD5 0a51db36b26916334170d525c5fbb840, while name_without_newline.txt (9 bytes) produced MD5 516771b62a75174bdc4454f19cfe3e35. The SHA-256 values were e755283b0993d48a... vs 4ece228218bc06bc... respectively. This confirms that even with longer strings, a single trailing newline character completely changes both hash values.

D.3 Hash Comparison Table

File	Bytes	MD5	SHA-256
hello_with_newline.txt	6	09f7e02f1290be211da707a266f153b3	66a045b452102c59d840ec097d59d9467e13a3f34f6494e539ffd32c1bb35f18
hello_without_newline.txt	5	8b1a9953c4611296a827abf8c4780d47	185f8db32271fe25f561a6fc938b2e264306ec304eda518007d176482631969
name_with_newline.txt	10	0a51db36b26916334170d525c5fbb840	e755283b0993d48a2643c09022d81e85e526681a46613797f0331dd02dc32154f
name_without_newline.txt	9	516771b62a75174bdc4454f19cfe3e35	4ece228218bc06bc517247048c93de6c2b4bea75bc4e6052fdc6db0e565241cc

To hash exactly the visible characters only, use echo -n or printf without a trailing newline. This ensures no extra 0x0A byte is included in the hash input. In forensic practice, always verify byte-level content with xxd before computing hashes.

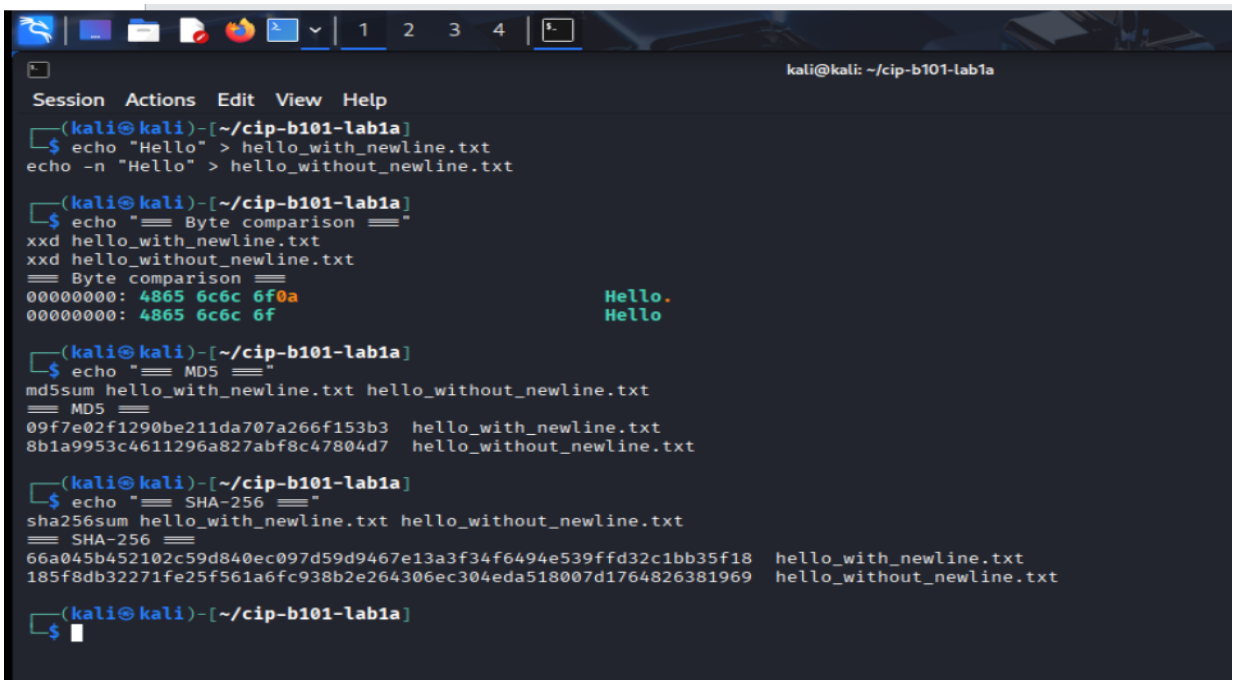

Figure 6: Hello hash comparison (Screenshot 6)

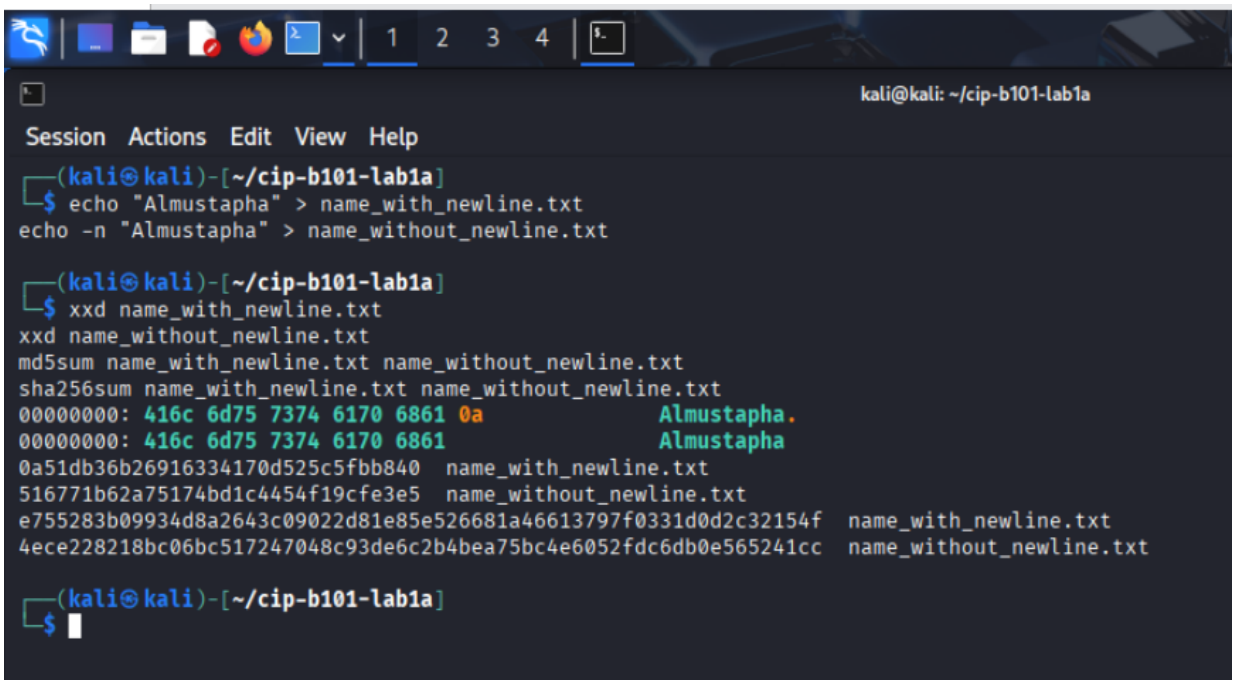

Figure 7: Name file hash comparison (Screenshot 7)

Task E: Endianness and Data Stored in Memory

Endianness describes the order in which bytes are stored in memory. In little-endian systems (most common in modern x86/x64 architectures), the least significant byte is stored at the lowest memory address. In big-endian systems (used in network protocols and some RISC architectures), the most significant byte comes first. A forensic analyst interpreting raw memory, file structures, or disk metadata must know which endianness applies, because the same four bytes can represent vastly different numerical values depending on byte order interpretation.

E.1 System Byte Order

Python reported sys.byteorder as "little", confirming that the Kali Linux system runs on a little-endian architecture (x86-64). This means that when the system stores a 32-bit integer in memory, the least significant byte occupies the lowest address.

E.2 Endianness Comparison Table

Raw Hex Bytes	Little-Endian Decimal	Big-Endian Decimal	Explanation
01000000	1	16,777,216	01 is LSB in LE, MSB in BE
FF000000	255	4,278,190,080	FF=255 in LSB position vs 4GB+ in MSB
00000100	65,536	256	01 byte at different positions
78563412	305,419,896	2,018,915,346	6.6x difference between interpretations

The same four bytes can produce different decimal values depending on byte order. For example, 01000000 interpreted as little-endian equals 1, but as big-endian equals 16,777,216. In forensic contexts, this matters when interpreting disk metadata (MBR partition tables use little-endian), network packets (big-endian per TCP/IP standards), and memory dumps. Misinterpreting endianness can lead to incorrect file size calculations, wrong timestamp readings, or invalid memory address resolution.

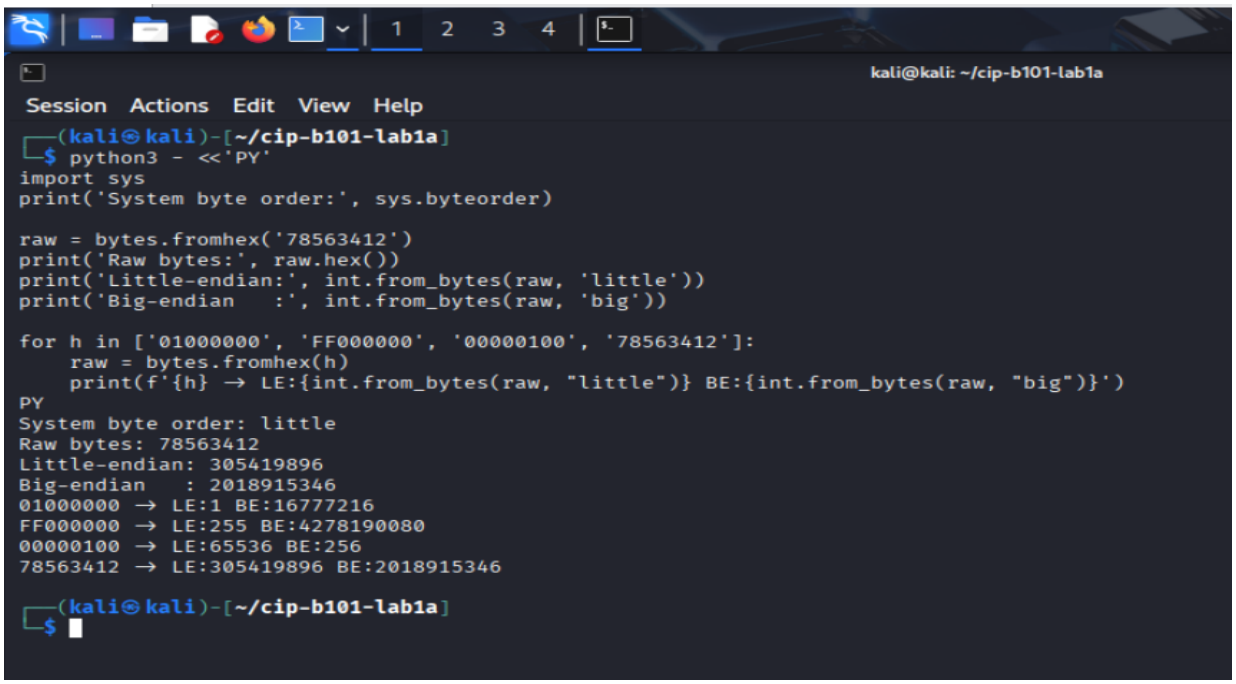

Figure 8: Endianness Python output (Screenshot 8)

Task F: Mini Forensic Interpretation Case

In this exercise, five evidence items were recovered from a training evidence source. Each item was decoded and interpreted using the techniques practiced in Tasks A through E. The interpretation demonstrates how number systems, ASCII encoding, timestamp conversion, and endianness awareness are applied in a practical forensic scenario.

Evidence Item	Value	Decoded Interpretation
EVID-001	48656c6c6f204943444641	Text: "Hello ICDFA" - hex decoded via bytes.fromhex()
EVID-002	01001000 01101001	Text: "Hi" - binary groups converted to ASCII (72=H, 105=i)
EVID-003	1675008832	Timestamp: 2023-01-29 16:13:52 UTC - Unix epoch conversion
EVID-004	725846400 (Mac)	Timestamp: 2024-01-02 00:00:00 UTC - Mac Absolute + 978307200s offset
EVID-005	78563412	LE: 305,419,896 / BE: 2,018,915,346 - endianness-dependent

F.1 Case Narrative

EVID-001 (48656c6c6f204943444641) was identified as an ASCII text string. Each pair of hex digits represents one byte: 48=H, 65=e, 6c=l, 6c=l, 6f=o, 20=space, 49=I, 43=C, 44=D, 46=F, 41=A. This item is useful for text and content interpretation, as it reveals a readable message that could be a marker, label, or identifier left in the evidence.

EVID-002 (01001000 01101001) contains two 8-bit binary values. Converting 01001000 to decimal yields 72 (ASCII H), and 01101001 yields 105 (ASCII i), spelling "Hi". This is a text/content interpretation item.

EVID-003 (1675008832) is a Unix epoch timestamp. Converting it using Python datetime.fromtimestamp() with UTC timezone produced 2023-01-29 16:13:52 UTC. This item is useful for timeline analysis, as it places an event at a specific moment in time.

EVID-004 (725846400) is a Mac Absolute Time value. Adding the 978,307,200-second Cocoa epoch offset yielded Unix timestamp 1704153600, which corresponds to 2024-01-02 00:00:00 UTC. This is also a timeline analysis item, requiring knowledge of the Mac epoch difference.

EVID-005 (78563412) demonstrates endianness ambiguity. As little-endian, the bytes represent 305,419,896 (0x12345678); as big-endian, they represent 2,018,915,346 (0x78563412). Without context about the source system architecture, both interpretations are valid. In forensic practice, the examiner must determine the correct byte order from the evidence source (e.g., x86 systems are LE, network data is typically BE) before assigning a definitive value.

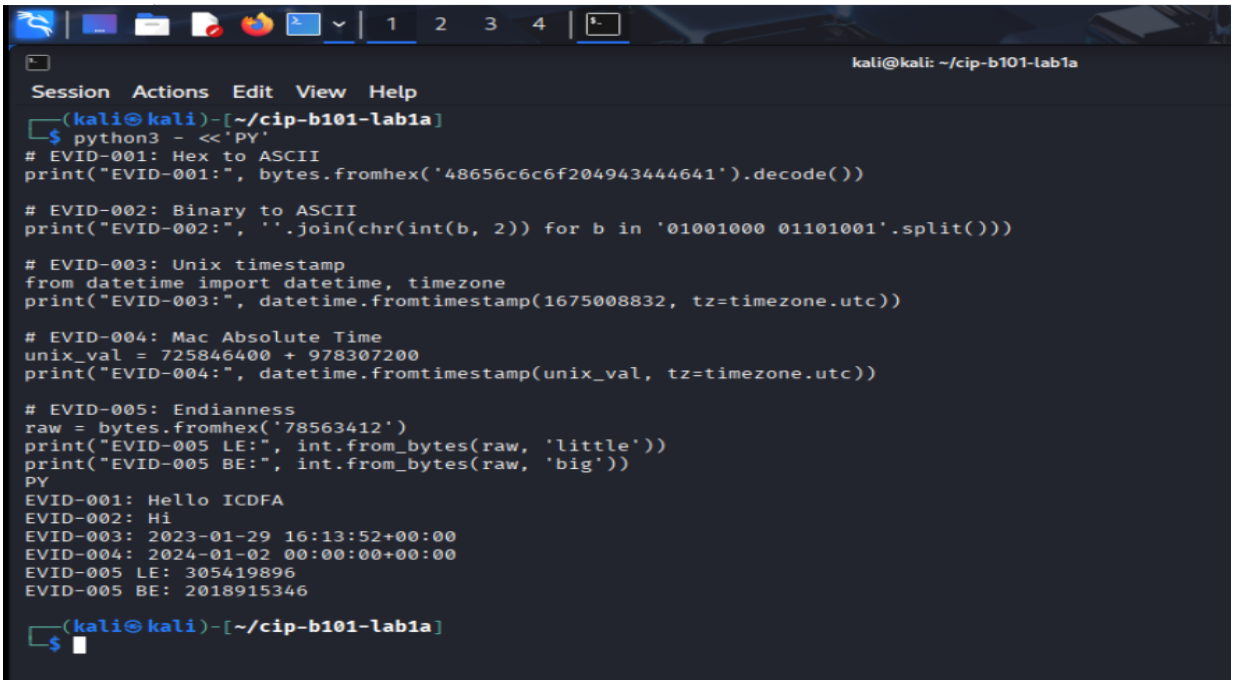

Figure 9: Forensic case interpretation commands (Screenshot 9)

Reflection

Number systems and timestamps are among the most fundamental concepts in digital forensics because they form the language through which all digital evidence speaks. Every file on a disk, every packet on a network, and every entry in a log is ultimately stored as binary or hexadecimal data. Without the ability to confidently convert between decimal, binary, and hexadecimal representations, a forensic analyst cannot correctly interpret file signatures, understand memory dumps, or validate hash values. Timestamps add another critical dimension: they provide the temporal context that transforms a collection of digital artifacts into a coherent forensic timeline. However, as this lab demonstrated, timestamps are not universal. Different operating systems use different epoch references (Unix vs. Mac Cocoa), and timezone misinterpretation can shift an event by hours, potentially altering the narrative of an investigation. The newline hashing exercise revealed how even a single invisible byte can compromise evidence integrity verification, while the endianness task showed that the same raw bytes can represent completely different values depending on the architecture. Together, these skills ensure that a forensic examiner does not merely run tools, but truly understands the data those tools reveal, which is the hallmark of a competent and credible digital forensics practitioner.

Screenshot Checklist

No.	Screenshot Evidence	Included?
1	Working folder creation and pwd output	Yes (Figure 1)
2	Binary/hex/decimal conversion commands	Yes (Figure 2)
3	xxd output for text evidence	Yes (Figure 3)
4	Hex-to-ASCII conversion output	Yes (Figure 4)
5	Unix timestamp conversion output	Yes (Figure 5)
6	Mac Absolute Time conversion output	Yes (Figure 5)
7	Hash comparison (md5sum and sha256sum)	Yes (Figures 6-7)
8	Endianness Python output	Yes (Figure 8)
9	Mini-case evidence interpretation commands	Yes (Figure 9)